



Compiler Dependency Inference

Q31

STATE (LEGACY) COMPILER

INTERVIEW QUESTION

Question. How does the Svelte compiler automatically infer dependencies for a legacy `$:` reactive statement?

ANSWER. In legacy Svelte (v3/v4), the compiler acts as a static analysis tool. When it encounters the `$:` label, it parses the trailing expression into an Abstract Syntax Tree (AST). It scans this tree to locate any top-level component variables that are read or evaluated within the statement. Svelte then automatically instruments the compiled JavaScript so that whenever any of those identified variables are reassigned, the reactive statement is queued to re-run.

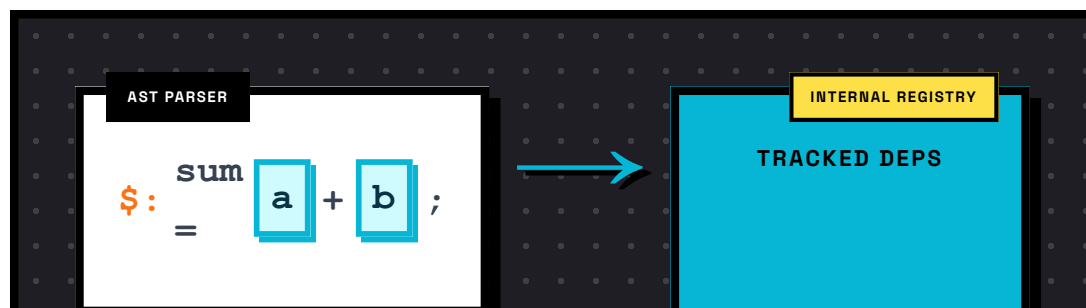
App.svelte

```
<script>
  let a = 1;
  let b = 2;
  let c = 3;

  // The compiler sees 'a' and 'b' being read.
  // It registers them as dependencies for this block.
  // 'c' is ignored because it is not evaluated here.
  $: sum = a + b;

  // For functions, the compiler only looks at the arguments passed or variables
  // explicitly read inside the `$:` line itself, NOT inside the function body!
  $: result = calculate(a); // Dependency: 'a' and 'calculate', but NOT variables inside
  'calculate'
</script>
```

① **Summary.** The legacy Svelte compiler relies on strict lexical analysis. If a top-level variable is visibly read inside the `$:` statement, it becomes a dependency. Variables hidden inside external function calls are completely invisible to the dependency tracker.





The diagram shows a large dark gray rectangle with a grid of small dots, representing a 2D array. A smaller, lighter gray rectangle is positioned in the top right corner of the large rectangle. Inside this lighter rectangle, a white square is outlined, and within it, a cyan square is outlined. The text "[a, b]" is written in white inside the white square. The entire diagram is framed by a thick black border on the left and right sides.

```
[ a, b ]
```